

Monte Carlo Tree Search: Gomoku Game

Xiaokun Chen, Hanyao Yu

Abstract

This project presents an implementation of the Gomoku game using the Monte Carlo Tree Search (MCTS) algorithm. The system model uses GomokuState, GomokuNode, GomokuMove, and GomokuMCTS. The MCTS algorithm is a very effective algorithm that uses thousands of simulations and backpropagation to find the best place to place the stone. To be specific, this algorithm implements the core concept: Selection, Expansion, Simulation, and Backpropagation. The selection makes the algorithm start at the root and use the upper confidence bound formula recursively. Then, when a leaf node is reached that is not terminal, the algorithm expands the node by generating its valid children. For the simulation, it will give an estimated result for how good that movement might be. For the backpropagation, the results from the simulation will be sent back to the tree, and the visit counts and win statistics will be updated.

Keywords: Monte Carlo Tree, Gomoku Game, Simulations

Rules of the Game:

This game must have two players, and they will use the black stone or the white stone to fight with each other. Black stone always goes first. If one player forms an unbroken line of five stones of their color horizontally, vertically, or diagonally, the player wins the game.

Run Game Guide:

1. Clone or download this repository: <https://github.com/Hanyao-Yu/DSAIPG>
2. Open this file with VSCODE, Eclipse, or another IDE that can run Java applications
3. You have to follow this path:
DSAIPG/src/main/java/com/phasmidsoftware/dsaipg/projects/mcts/gomoku/Main.java, find Main.java, and click the run button to run this game.

Project Background

With AI technology advancing so fast, board games have become a key way to test an AI's "intelligence." The famous AlphaGo matches showed that Monte Carlo Tree Search (MCTS), together with deep learning, can beat top human players in very complex games like Go. Gomoku, by comparison, has simpler rules and uses a smaller board (for example, 9×9), yet it still offers deep strategic challenges and is a worthwhile subject for study.

Main System Architecture

GomokuState

Manages the board itself. It can:

- a. Spot an immediate win (or block your opponent if they're about to win) via `findImmediateWin`.
- b. Produce a list of all legal moves with `moves()`.
- c. Pick a move during playouts—still checking for instant wins or blocks—using `chooseMove()`.
- d. Advance the board to the next state and check if the game has ended (`next()` and `isTerminal()`).

GomokuNode

Represents one node in the MCTS search tree. Each node:

- a. Knows its parent and children.

- b. Tracks how many times it's been visited and how often it led to a win.
- c. Expands its children lazily the first time you call children().
- d. Propagates simulation results back up the tree with record().

GomokuMCTS

Drives the Monte Carlo Tree Search in five steps:

Selection: Descend the tree using the UCT formula.

Expansion: At a leaf node, call children() to add all possible next moves.

Simulation: Play out a random game (with win/block shortcuts) to the end.

Backpropagation: Feed the result back up via record().

Move Selection: Choose the child node with the most visits and promote it as the new root.

GomokuMove

This class is to describe how stone can be moved.

- a. public GomokuMove(int row, int col, int player), we used row and col to describe the row and column on the board.
- b. We override equal method to check whether the player put the stone in the same place

Gomoku

This class is to define who will start first, and decide the size of the board, for example, 9 x 9 or 15 x 15. Additionally, this class also defines how many stones in a row the players can win.

- a. Start method to decide the board and the beginner.
- b. public static final int WIN to define how many stones in a row to win

Experiment Design

a.Board size: 9×9

b.Opening move: black always goes first

c.Iteration settings:

1. White 500 vs. Black 500
2. White 500 vs. Black 1000
3. White 500 vs. Black 2000

4.White 500 vs. Black 5000

5.White 500 vs. Black 10000

d.Performance metrics:

1.Time per move (ms per move)

2.Total game time (sum of all per-move times)

3.Average time per game

4.Average move per game

Game demo on a 9×9 board: Player 0 ran 20,000 iterations while Player 1 ran 10,000 iterations. After 19 moves, Player 0 wins.

```
yuhanyao@Hanyaos-MacBook-Pro DSAIPG % cd /Users/yuhanyao/INFO6205/DSAIPG ; /usr/bin/echo
olders/c9/r8j108hd5yx32pbymhv0vs480000gn/T/cp_d2kjb6ygo7n3ntt9nbjs39e9.argfile com.ph
Board:
  0 1 2 3 4 5 6 7 8
0 . . . . . . . . .
1 . . . . . . . . .
2 . . . . . . . . .
3 . . . . . . . . .
4 . . . . . . . . .
5 . . . . . . . . .
6 . . . . . . . . .
7 . . . . . . . . .
8 . . . . . . . . .

Move #1: Player 0 (iters=20000) plays: [0:(4,4)] // step time: 2381 ms
Board:
  0 1 2 3 4 5 6 7 8
0 . . . . . . . . .
1 . . . . . . . . .
2 . . . . . . . . .
3 . . . . . . . . .
4 . . . . O . . . .
5 . . . . . . . . .
6 . . . . . . . . .
7 . . . . . . . . .
8 . . . . . . . . .

Move #2: Player 1 (iters=10000) plays: [1:(2,4)] // step time: 1131 ms
Board:
  0 1 2 3 4 5 6 7 8
0 . . . . . . . . .
1 . . . . . . . . .
2 . . . . ● . . . .
3 . . . . . . . . .
4 . . . . O . . . .
5 . . . . . . . . .
6 . . . . . . . . .
7 . . . . . . . . .
8 . . . . . . . . .
```

Move #3: Player 0 (iters=20000) plays: [0:(0,4)] // step time: 2160 ms

Board:

```
  0 1 2 3 4 5 6 7 8
0 . . . . . ○ . . . .
1 . . . . . . . . . .
2 . . . . . ● . . . .
3 . . . . . . . . . .
4 . . . . . ○ . . . .
5 . . . . . . . . . .
6 . . . . . . . . . .
7 . . . . . . . . . .
8 . . . . . . . . . .
```

Move #4: Player 1 (iters=10000) plays: [1:(7,3)] // step time: 1047 ms

Board:

```
  0 1 2 3 4 5 6 7 8
0 . . . . . ○ . . . .
1 . . . . . . . . . .
2 . . . . . ● . . . .
3 . . . . . . . . . .
4 . . . . . ○ . . . .
5 . . . . . . . . . .
6 . . . . . . . . . .
7 . . . ● . . . . .
8 . . . . . . . . . .
```

Move #5: Player 0 (iters=20000) plays: [0:(5,4)] // step time: 2022 ms

Board:

```
  0 1 2 3 4 5 6 7 8
0 . . . . . ○ . . . .
1 . . . . . . . . . .
2 . . . . . ● . . . .
3 . . . . . . . . . .
4 . . . . . ○ . . . .
5 . . . . . ○ . . . .
6 . . . . . . . . . .
7 . . . ● . . . . .
8 . . . . . . . . . .
```

Move #6: Player 1 (iters=10000) plays: [1:(1,2)] // step time: 987 ms

Board:

```
  0 1 2 3 4 5 6 7 8
0 . . . . . O . . . .
1 . . ● . . . . . .
2 . . . . . ● . . . .
3 . . . . . . . . . .
4 . . . . . O . . . .
5 . . . . . O . . . .
6 . . . . . . . . . .
7 . . . . ● . . . . .
8 . . . . . . . . . .
```

Move #7: Player 0 (iters=20000) plays: [0:(6,4)] // step time: 2002 ms

Board:

```
  0 1 2 3 4 5 6 7 8
0 . . . . . O . . . .
1 . . ● . . . . . .
2 . . . . . ● . . . .
3 . . . . . . . . . .
4 . . . . . O . . . .
5 . . . . . O . . . .
6 . . . . . O . . . .
7 . . . . ● . . . . .
8 . . . . . . . . . .
```

Move #8: Player 1 (iters=10000) plays: [1:(7,4)] // step time: 915 ms

Board:

```
  0 1 2 3 4 5 6 7 8
0 . . . . . O . . . .
1 . . ● . . . . . .
2 . . . . . ● . . . .
3 . . . . . . . . . .
4 . . . . . O . . . .
5 . . . . . O . . . .
6 . . . . . O . . . .
7 . . . . ● ● . . . .
8 . . . . . . . . . .
```

Move #9: Player 0 (iters=20000) plays: [0:(6,6)] // step time: 1828 ms

Board:

```
  0 1 2 3 4 5 6 7 8
0 . . . . . ○ . . . .
1 . . ● . . . . . .
2 . . . . . ● . . . .
3 . . . . . . . . . .
4 . . . . . ○ . . . .
5 . . . . . ○ . . . .
6 . . . . . ○ . ○ . .
7 . . . . ● . . . . .
8 . . . . . . . . . .
```

Move #10: Player 1 (iters=10000) plays: [1:(5,7)] // step time: 928 ms

Board:

```
  0 1 2 3 4 5 6 7 8
0 . . . . . ○ . . . .
1 . . ● . . . . . .
2 . . . . . ● . . . .
3 . . . . . . . . . .
4 . . . . . ○ . . . .
5 . . . . . ○ . . ● .
6 . . . . . ○ . ○ . .
7 . . . . ● . . . . .
8 . . . . . . . . . .
```

Move #11: Player 0 (iters=20000) plays: [0:(6,5)] // step time: 1724 ms

Board:

```
  0 1 2 3 4 5 6 7 8
0 . . . . . ○ . . . .
1 . . ● . . . . . .
2 . . . . . ● . . . .
3 . . . . . . . . . .
4 . . . . . ○ . . . .
5 . . . . . ○ . . ● .
6 . . . . . ○ ○ ○ . .
7 . . . . ● . . . . .
8 . . . . . . . . . .
```

Move #12: Player 1 (iters=10000) plays: [1:(6,3)] // step time: 800 ms

Board:

```
  0 1 2 3 4 5 6 7 8
0 . . . . ○ . . . .
1 . . ● . . . . . .
2 . . . . ● . . . .
3 . . . . . . . . .
4 . . . . ○ . . . .
5 . . . . ○ . . ● .
6 . . . ● ○ ○ ○ . .
7 . . . ● ● . . . .
8 . . . . . . . . .
```

Move #13: Player 0 (iters=20000) plays: [0:(6,8)] // step time: 1621 ms

Board:

```
  0 1 2 3 4 5 6 7 8
0 . . . . ○ . . . .
1 . . ● . . . . . .
2 . . . . ● . . . .
3 . . . . . . . . .
4 . . . . ○ . . . .
5 . . . . ○ . . ● .
6 . . . ● ○ ○ ○ ○ ○
7 . . . ● ● . . . .
8 . . . . . . . . .
```

Move #14: Player 1 (iters=10000) plays: [1:(6,7)] // step time: 732 ms

Board:

```
  0 1 2 3 4 5 6 7 8
0 . . . . ○ . . . .
1 . . ● . . . . . .
2 . . . . ● . . . .
3 . . . . . . . . .
4 . . . . ○ . . . .
5 . . . . ○ . . ● .
6 . . . ● ○ ○ ○ ● ○
7 . . . ● ● . . . .
8 . . . . . . . . .
```


Move #15: Player 0 (iters=20000) plays: [0:(4,3)] // step time: 1601 ms

Board:

```
  0 1 2 3 4 5 6 7 8
0 . . . . ○ . . . .
1 . . ● . . . . . .
2 . . . . ● . . . .
3 . . . . . . . . .
4 . . . ○ ○ . . . .
5 . . . . ○ . . ● .
6 . . . ● ○ ○ ○ ● ○
7 . . . ● ● . . . .
8 . . . . . . . . .
```

Move #16: Player 1 (iters=10000) plays: [1:(2,1)] // step time: 766 ms

Board:

```
  0 1 2 3 4 5 6 7 8
0 . . . . ○ . . . .
1 . . ● . . . . . .
2 . ● . . ● . . . .
3 . . . . . . . . .
4 . . . ○ ○ . . . .
5 . . . . ○ . . ● .
6 . . . ● ○ ○ ○ ● ○
7 . . . ● ● . . . .
8 . . . . . . . . .
```

Move #17: Player 0 (iters=20000) plays: [0:(7,6)] // step time: 1445 ms

Board:

```
  0 1 2 3 4 5 6 7 8
0 . . . . ○ . . . .
1 . . ● . . . . . .
2 . ● . . ● . . . .
3 . . . . . . . . .
4 . . . ○ ○ . . . .
5 . . . . ○ . . ● .
6 . . . ● ○ ○ ○ ● ○
7 . . . ● ● . ○ . .
8 . . . . . . . . .
```

```
Move #18: Player 1 (iters=10000) plays: [1:(8,7)] // step time: 601 ms
```

```
Board:
```

```
  0 1 2 3 4 5 6 7 8
0 . . . . . O . . . .
1 . . ● . . . . . . .
2 . ● . . . ● . . . .
3 . . . . . . . . . .
4 . . . . O O . . . .
5 . . . . . O . . ● .
6 . . . ● O O O ● O
7 . . . ● ● . O . .
8 . . . . . . . ● .
```

```
Move #19: Player 0 (iters=20000) plays: [0:(3,2)] // step time: 599 ms
```

```
Board:
```

```
  0 1 2 3 4 5 6 7 8
0 . . . . . O . . . .
1 . . ● . . . . . . .
2 . ● . . . ● . . . .
3 . . O . . . . . . .
4 . . . . O O . . . .
5 . . . . . O . . ● .
6 . . . ● O O O ● O
7 . . . ● ● . O . .
8 . . . . . . . ● .
```

```
Game over. Total game time: 25328 ms
```

```
Winner is Player 0!
```

```
yuhanyao@Hanyaos-MacBook-Pro DSAIPG %
```

We fixed White's iteration count at 500 and ran Black with {500, 1,000, 2,000, 5,000, 10,000} iterations. After ten games for each setting, we recorded Black's win rate, White's win rate, the average number of moves per game, and the average time per game, as shown below.

```

yuhanyao@Hanyaos-MacBook-Pro DSAIPG % /usr/bin/env /Library/Java/JavaVirtualMa
0gn/T/cp_d2kjpg6ygo7n3ntt9nbjs39e9.argfile com.phasmidsoftware.dsaipg.projects.
Running Gomoku simulation with different iteration counts
=====

Configuration: WHITE_ITERS=500, BLACK_ITERS=500 (10 games)
-----

Results:
White wins: 3 (30.0%)
Black wins: 7 (70.0%)
Draws: 0 (0.0%)
Average game duration: 929.9 ms
Average moves per game: 22.7
=====

Configuration: WHITE_ITERS=500, BLACK_ITERS=1000 (10 games)
-----

Results:
White wins: 3 (30.0%)
Black wins: 7 (70.0%)
Draws: 0 (0.0%)
Average game duration: 1335.1 ms
Average moves per game: 22.7
=====

Configuration: WHITE_ITERS=500, BLACK_ITERS=2000 (10 games)
-----

Results:
White wins: 2 (20.0%)
Black wins: 8 (80.0%)
Draws: 0 (0.0%)
Average game duration: 2105.2 ms
Average moves per game: 19.8
=====

Configuration: WHITE_ITERS=500, BLACK_ITERS=5000 (10 games)
-----

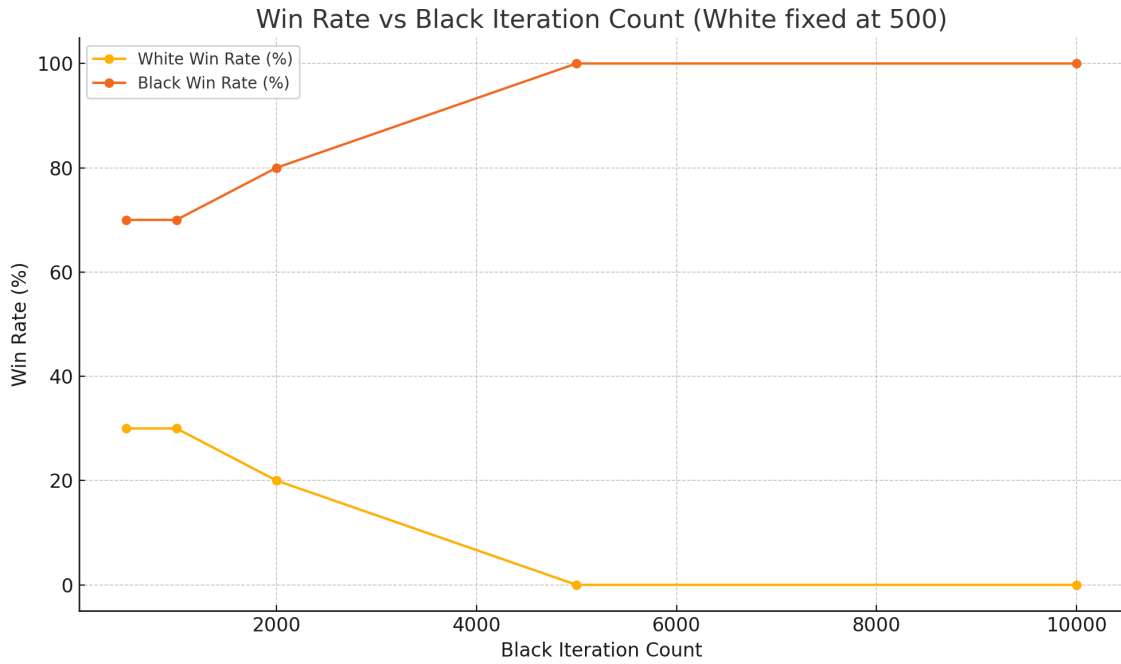
Results:
White wins: 0 (0.0%)
Black wins: 10 (100.0%)
Draws: 0 (0.0%)
Average game duration: 3280.6 ms
Average moves per game: 13.2
=====

Configuration: WHITE_ITERS=500, BLACK_ITERS=10000 (10 games)
-----

Results:
White wins: 0 (0.0%)
Black wins: 10 (100.0%)
Draws: 0 (0.0%)
Average game duration: 5919.6 ms
Average moves per game: 12.8
=====
yuhanyao@Hanyaos-MacBook-Pro DSAIPG %

```

Based on the above experiment, we made a graph below:



Based on the above graph, we can see that with the increase of the iteration (simulation) count, the win rate will also increase. We fixed white at 500, and after 5000 iterations of black, the win rate of black will become 100%, and white stone cannot win anymore.

Optimization

Our current Monte Carlo Tree Search for Gomoku is basic, we just used selection using UCT, expansion through lazy generation of child nodes, playout via uniform random simulations, and backpropagation to update node statistics. However, this does have some weaknesses because the simulations are purely random. Therefore, we decided to do some optimization. We want to give MCTS some limitations that can make it more strategic or human-like gameplay.

Inspired by human thinking patterns, we discovered that players tend to put stones near their previous ones, and when the opponent gets three stones in one line, they will try to block the way of opponent. Therefore, we enhanced our global awareness heuristic, which can enable AI to recognize threats and the chance of winning. These enhancements make the playouts more informed and can improve the accuracy and convergence of MCTS under limited iterations.

We add a method named **detectThreats** to help the system decide whether it needs to defend or attack, to be specific, if the opponent gets three or four in one line, the system will be based on the “threats” score to block the enemy. Additionally, we also add a method to put the first stone in the middle of the board, which makes the AI more strategic and human-like.

After we implement the optimization:

```
TERMINAL Run: Ma
aVirtualMachines/jdk-22.jdk/Contents/Home/bin/java @/var/folders/c9/r8jl08hd5yx32pbymhv0vs480000
kjp6ygo7n3ntt9nbjs39e9.argfile com.phasmidsoftware.dsaipg.projects.mcts.gomoku.Main
Running Gomoku simulation with different iteration counts
=====

Configuration: WHITE_ITS=500, BLACK_ITS=500 (10 games)
=====

Results:
White wins: 6 (60.0%)
Black wins: 4 (40.0%)
Draws: 0 (0.0%)
Average game duration: 6820.1 ms
Average moves per game: 17.4
=====

Configuration: WHITE_ITS=500, BLACK_ITS=1000 (10 games)
=====

Results:
White wins: 2 (20.0%)
Black wins: 8 (80.0%)
Draws: 0 (0.0%)
Average game duration: 10721.6 ms
Average moves per game: 20.4
=====

Configuration: WHITE_ITS=500, BLACK_ITS=2000 (10 games)
=====

Results:
White wins: 1 (10.0%)
Black wins: 9 (90.0%)
Draws: 0 (0.0%)
Average game duration: 14294.1 ms
Average moves per game: 14.5
=====

Configuration: WHITE_ITS=500, BLACK_ITS=5000 (10 games)
=====

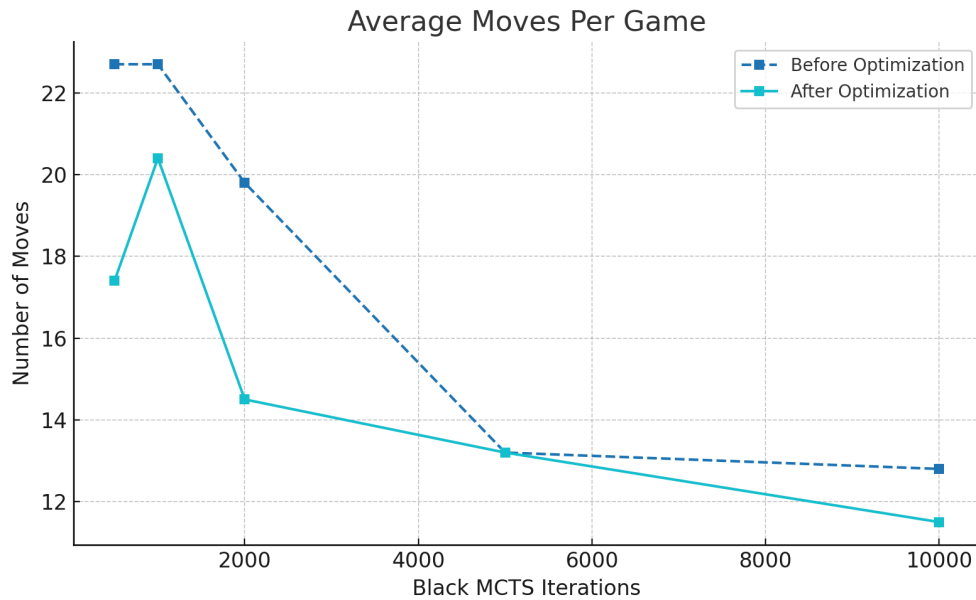
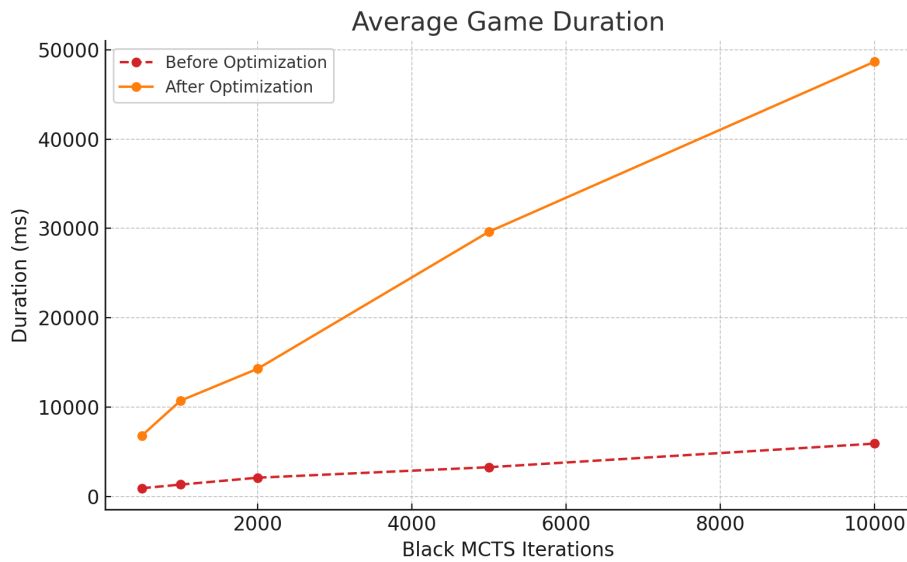
Results:
White wins: 0 (0.0%)
Black wins: 10 (100.0%)
Draws: 0 (0.0%)
Average game duration: 29633.6 ms
Average moves per game: 13.2
=====

Configuration: WHITE_ITS=500, BLACK_ITS=10000 (10 games)
=====

Results:
White wins: 1 (10.0%)
Black wins: 9 (90.0%)
Draws: 0 (0.0%)
Average game duration: 48659.2 ms
Average moves per game: 11.5
=====
```

	A	B	C
	Game_Duration_Comparison		
1	Black Iterations	Duration Before (ms)	Duration After (ms)
2	500	929.9	6820.1
3	1000	1335.1	10721.6
4	2000	2105.2	14294.1
5	5000	3280.6	29633.6
6	10000	5919.6	48659.2

	A	B	C
	Average_Moves_Comparison		
1	Black Iterations	Avg Moves Before	Avg Moves After
2	500	22.7	17.4
3	1000	22.7	20.4
4	2000	19.8	14.5
5	5000	13.2	13.2
6	10000	12.8	11.5



Based on the above table, we can see that after optimization, the run time increased significantly, but the average movement decreased. Although each calculation takes more time, every move becomes significantly more valuable. Now, the system will not play the game randomly, it will find the winning opportunities quickly and defend correctly.

All in all, with the increase of simulations and optimizations, the runtime increased dramatically by nearly ten times in some cases. However, this result is expected because every step becomes more and more complex. To be specific, such as move sorting, threat detection, and positional evaluation. These additional methods enable the computer to think more, and it needs more time to put the stone in a tactically advantageous position and ultimately improve its win rate.

Conclusion

1. **Iteration vs. win rate**

Using the same MCTS algorithm, the win rate improves as you raise the iteration count. However, beyond a certain point (for example, comparing 500 vs. 5,000 iterations), the win rate hardly changes, while the time cost keeps climbing.

2. **Local move pruning**

By limiting candidate moves to within a radius of 2 around the previous move, we can fit more simulations into the same time budget and cut down on per-move decision time.

3. **“Block fours” in simulation**

Adding a simple “always block four-in-a-row” rule during rollout gives a big jump in both decision efficiency and overall win rate.

Overall, our work shows that although more simulations can increase the win rate. However, it is not a smart method to just increase the simulations to get a higher win rate. Because you can see that after 5000 simulations, the win rate will not increase significantly, while the time cost keeps climbing. This means that indefinitely increasing the number of simulation times is not cost-effective, as it leads to diminishing returns with extremely low efficiency gains.

Unit Test for Tictactoe:

The image displays two screenshots of an IDE (Visual Studio Code) showing unit tests for a TicTacToe application. The first screenshot shows the `MCTSTest.java` file, and the second screenshot shows the `PositionTest.java` file. Both files are part of the `com.phasmidsoftware.dsaipg.projects.mcts.tictactoe` package.

First Screenshot: `MCTSTest.java`

```
src > test > java > com > phasmidsoftware > dsaipg > projects > mcts > tictactoe > J MCTSTest.java > ...
1 package com.phasmidsoftware.dsaipg.projects.mcts.tictactoe;
2
3 import static org.junit.Assert.assertEquals;
4 import static org.junit.Assert.assertNotNull;
5 import static org.junit.Assert.assertTrue;
6 import org.junit.Test;
7
8 import com.phasmidsoftware.dsaipg.projects.mcts.core.Node;
9
10 public class MCTSTest {
11     @Test
12     public void testBestMoveAfterSimulation() {
13         TicTacToe.TicTacToeState initialState = new TicTacToe().new TicTacToeState();
14         TicTacToeNode root = new TicTacToeNode(initialState);
15         MCTS mcts = new MCTS(root);
16
17         // run 1000 simulations
18         for (int i = 0; i < 1000; i++) {
19             mcts.runSimulation(root);
20         }
21
22         Node<TicTacToe> best = mcts.bestChild(root);
23
24         // check that we found a valid state
25         assertNotNull(best);
26     }
27 }
```

The `TEST RESULTS` panel shows the following tests passed:

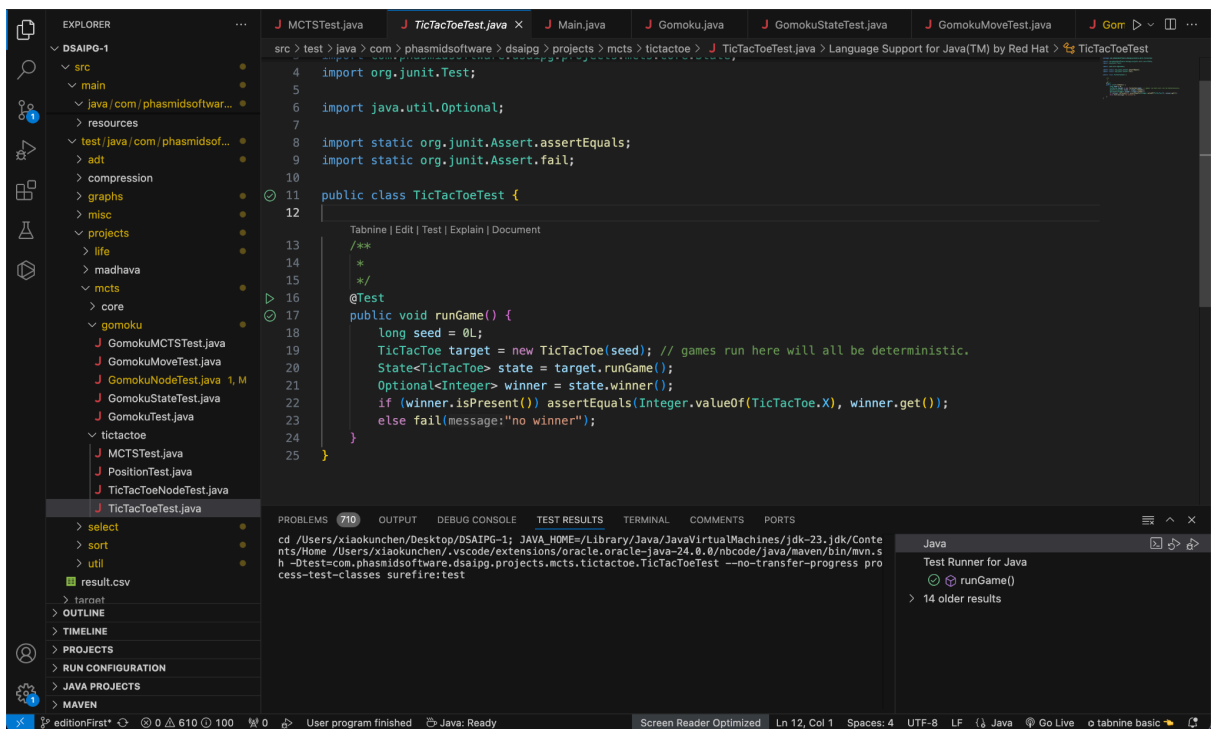
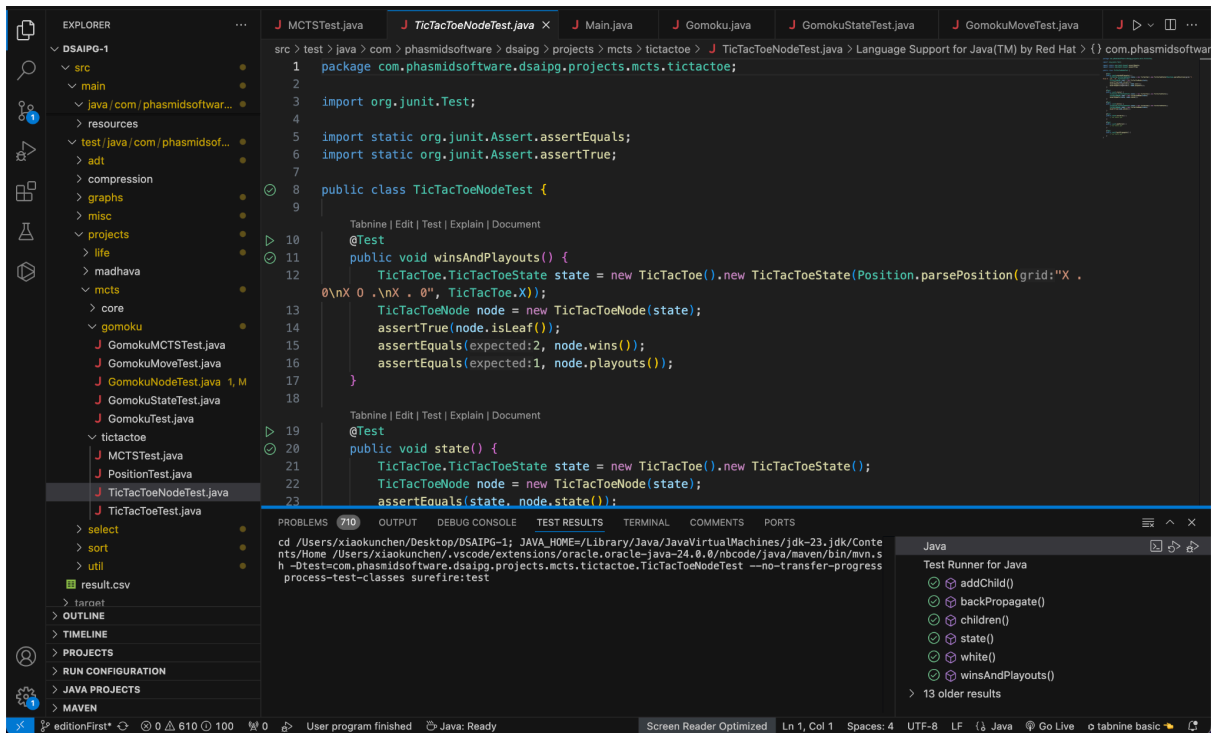
- testBestMoveAfterSimulation()
- testRunSimulationIncreasesPlayouts()
- testSimulationReturnsValidScore()
- 11 older results

Second Screenshot: `PositionTest.java`

```
src > test > java > com > phasmidsoftware > dsaipg > projects > mcts > tictactoe > J PositionTest.java > Language Support for Java(TM) by Red Hat > {} com.phasmidsoftware.dsaipg
1 package com.phasmidsoftware.dsaipg.projects.mcts.tictactoe;
2
3 import org.junit.Test;
4
5 import java.util.List;
6 import java.util.Optional;
7
8 import static org.junit.Assert.*;
9
10 public class PositionTest {
11
12 }
```

The `TEST RESULTS` panel shows the following tests passed:

- testFull()
- testMove0()
- testMove1()
- testMove_1()
- testMove_2()
- testMoves()
- testParseCell()
- testProjectCol()
- testProjectDiag()
- testProjectRow()
- testReflect()
- testRender()
- testRotate()
- testThreeInARow()
- testToString()
- testWinner0()
- testWinner1()
- testWinner2()
- 12 older results



Unit Test for Gomoku:

This screenshot shows the VS Code editor with the file `GomokuMCTSTest.java` open. The Explorer sidebar on the left shows the project structure for `DSAIPG-1`, with the `test/java/com/phasmidsoftware/mcts/gomoku` directory selected. The editor displays the following code:

```
1 package com.phasmidsoftware.dsaipg.projects.mcts.gomoku;
2
3 import static org.junit.Assert.assertEquals;
4 import static org.junit.Assert.assertNotEquals;
5 import static org.junit.Assert.assertNotNull;
6 import static org.junit.Assert.assertTrue;
7 import org.junit.Test;
8
9 import com.phasmidsoftware.dsaipg.projects.mcts.core.Move;
10
11 public class GomokuMCTSTest {
12     @Test
13     public void testFindNextMoveIsNotNull() {
14         GomokuState state = new GomokuState(new int[Gomoku.SIZE][Gomoku.SIZE], player:0);
15         GomokuNode root = new GomokuNode(state);
16         GomokuMCTS mcts = new GomokuMCTS(root);
17
18         Move<Gomoku> move = mcts.findNextMove(iterations:100);
19         assertNotNull(move);
20     }
21 }
```

The bottom panel shows the **TEST RESULTS** tab with the following tests passed:

- testFindNextMoveIsNotNull()
- testFindNextMoveChoosesWinWhenAvailable()
- testFindNextMoveUpdatesRoot()
- testUCTSelectionPrefersHighWinRate()

5 older results are also listed.

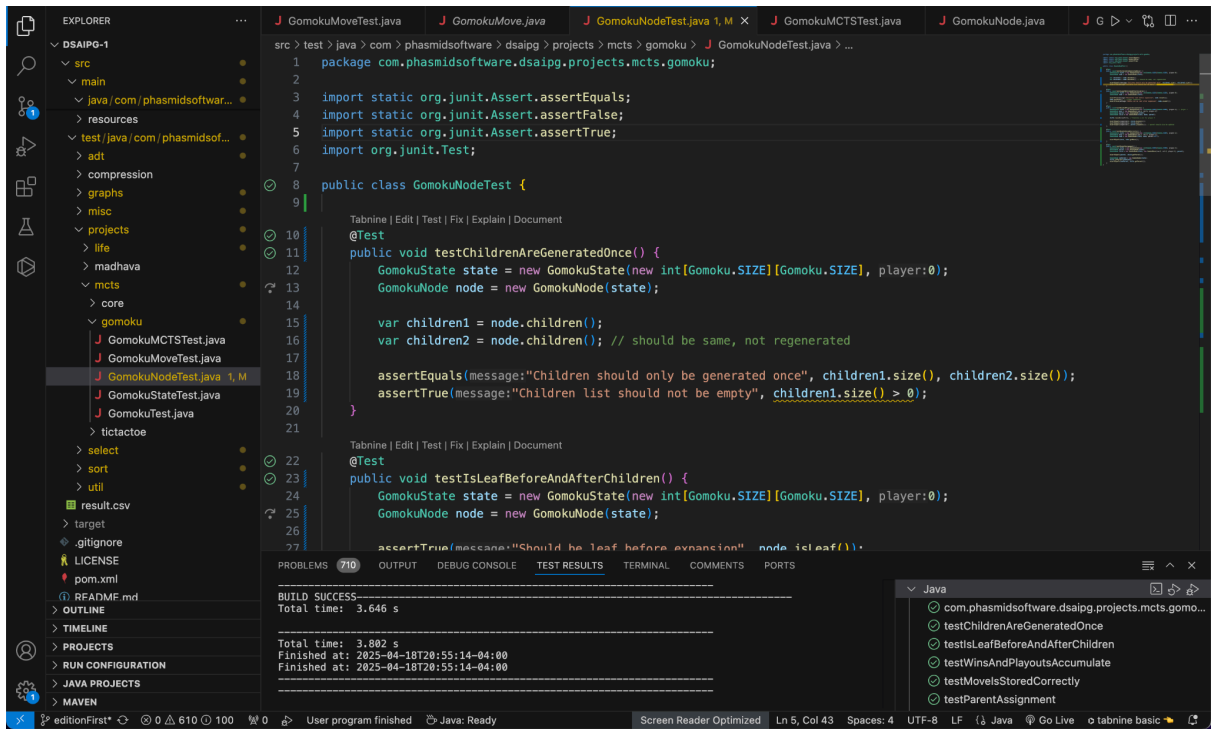
This screenshot shows the VS Code editor with the file `GomokuMoveTest.java` open. The Explorer sidebar on the left shows the project structure for `DSAIPG-1`, with the `test/java/com/phasmidsoftware/mcts/gomoku` directory selected. The editor displays the following code:

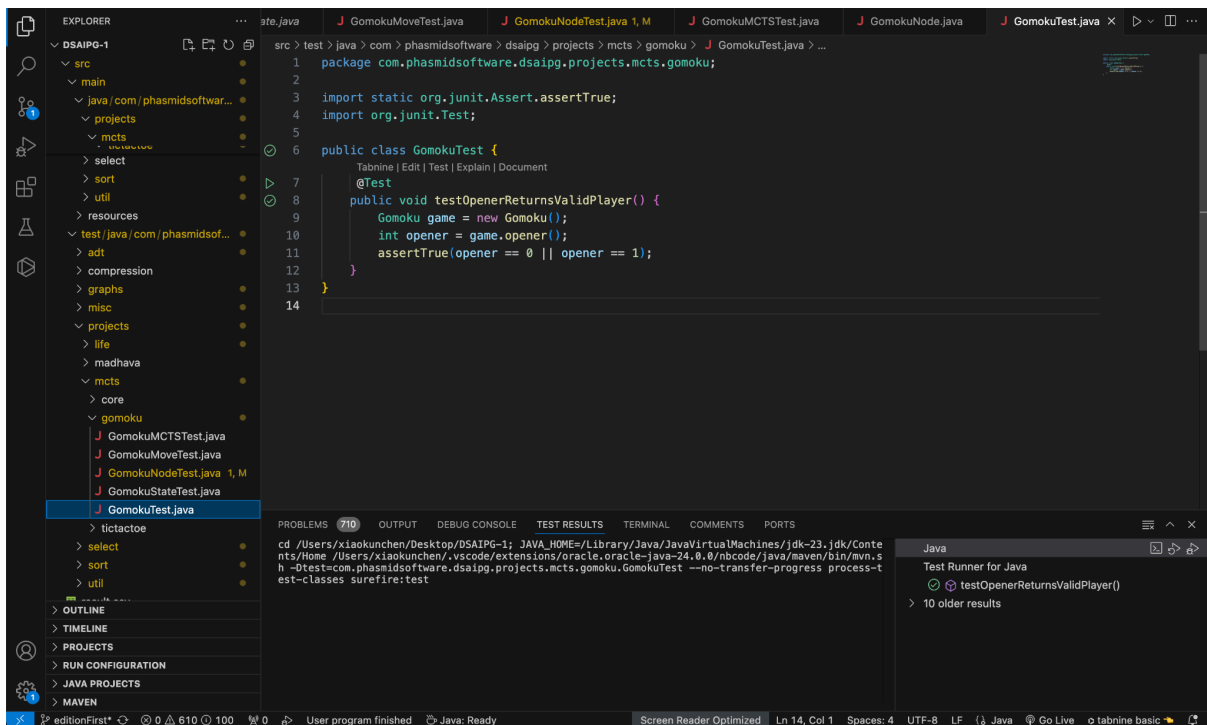
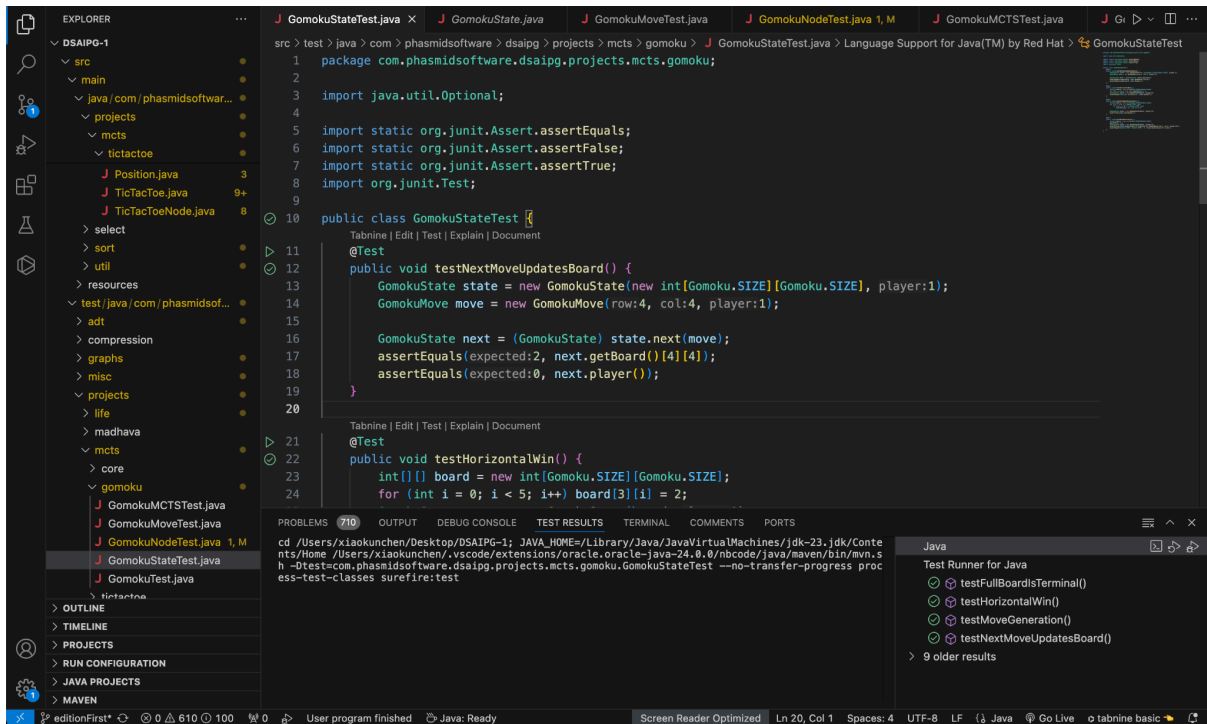
```
1 package com.phasmidsoftware.dsaipg.projects.mcts.gomoku;
2
3 import static org.junit.Assert.assertEquals;
4 import static org.junit.Assert.assertNotEquals;
5 import org.junit.Test;
6
7 public class GomokuMoveTest {
8     @Test
9     public void testEqualsAndHashCode() {
10         GomokuMove m1 = new GomokuMove(row:1, col:2, player:0);
11         GomokuMove m2 = new GomokuMove(row:1, col:2, player:0);
12         GomokuMove m3 = new GomokuMove(row:1, col:2, player:1);
13
14         assertEquals(m1, m2);
15         assertEquals(m1.hashCode(), m2.hashCode());
16         assertNotEquals(m1, m3);
17     }
18
19     @Test
20     public void testToStringFormat() {
21         GomokuMove move = new GomokuMove(row:3, col:5, player:1);
22         assertEquals(expected:"1:(3,5)", move.toString());
23     }
24 }
```

The bottom panel shows the **TEST RESULTS** tab with the following tests passed:

- testEqualsAndHashCode()
- testToStringFormat()

6 older results are also listed.





Reference:

1. Gomoku. (n.d.). In *Wikipedia*. Retrieved April 18, 2025, from <https://en.wikipedia.org/wiki/Gomoku>
2. Monte Carlo tree search. (n.d.). *Wikipedia*. Retrieved April 18, 2025, from https://en.wikipedia.org/wiki/Monte_Carlo_tree_search
3. Int8. (2018, March 23). *Monte Carlo Tree Search – beginners guide*. Int8.io. <https://int8.io/monte-carlo-tree-search-beginners-guide/>